CS6600_18_PA1

Deepak Charan S, Agnideb Mukherjee, Sooryanarayanan G

1 Introduction

This assignment aims at building a simple single-issue, in-order five-stage pipeline processor simulator.

The simulator reads commands from standard input, and displays the intermediate structure and statistics with various commands (also from std-input). This machine is `Harvard-Architecture` which means instructions and data both live in two separate memories. Therefore an instruction at some address, and data at the same address will not collide.

2 Design Specifications

There are mainly three files on which the entire simulator is built; `pipesim.h`, which defines the framework API and the ISA, `framework.cc`, which implements the parser, command loop, builder, validation and the clock engine, and, `design.cc`, which is the processor itself.

2.1 ISA

There are eight 32-bit register, x_0 to x_7. The supported opcodes are **ADD**, **SUB**, **AND**, **OR**, **XOR**, **ADDI**, **LW**, **SW**, **BEQ**, **BNE**, **J**, and, **HALT**. Every instruction is of the format

INST <pc> <opcode> <rd> <rs1> <rs2> <immediate>

2.2 Pipeline

There are five stages: **IF**, **ID**, **EX**, **MEM** and **WB**. The processor design is in such a way that its single-issue, and in-order. There are no forwarding paths, no branch guessing or any other complex mechanisms. On a hazard, the pipeline is just stalled until it gets resolved.

All the stages reads the current pipeline state. The framework evaluates stages in the reverse order, i.e. from `WB` to `IF`. This is because, in hardware everything

runs in parallel, but a software code is executed sequentially. If we run it sequentially with **IF** $\rightarrow$ **WB**, then a newer instruction in an earlier stage might read old value from a register, which was supposed to be modified by an older instruction in a later stage. By running it in reverse order, we ensure that there types of errors don't happen, and the simulation is functionally equivalent to that of actual hardware implementation.

2.3 HazardControl

The Hazard-control unit detects read-after-write (RAW) hazards by comparing the source registers of the instruction in the IF/ID latch with the destination registers of older instructions in the ID/EX and EX/MEM latches.

It first checks whether the current opcode actually reads rs1 and/or rs2 (avoiding false dependencies for instructions such as ADDI, LW, J, HALT, and NOP). A hazard is declared only when a valid older instruction writes a non-zero destination register that matches a source register of the decoding instruction.

When a data hazard is found, the processor stalls the front end; the PC and IF/ID latch are held so that the dependent instruction remains in the decode stage, while a bubble is inserted into ID/EX. Older instructions continue through EX, MEM, and WB normally.

The design also follows the required WB-before-ID timing, allowing decode to see a value written in the same clock cycle.

2.4 Branch Resolution:

For control hazards, branch and jump instructions are identified in the decode stage, but their final outcome is resolved in the execute stage.

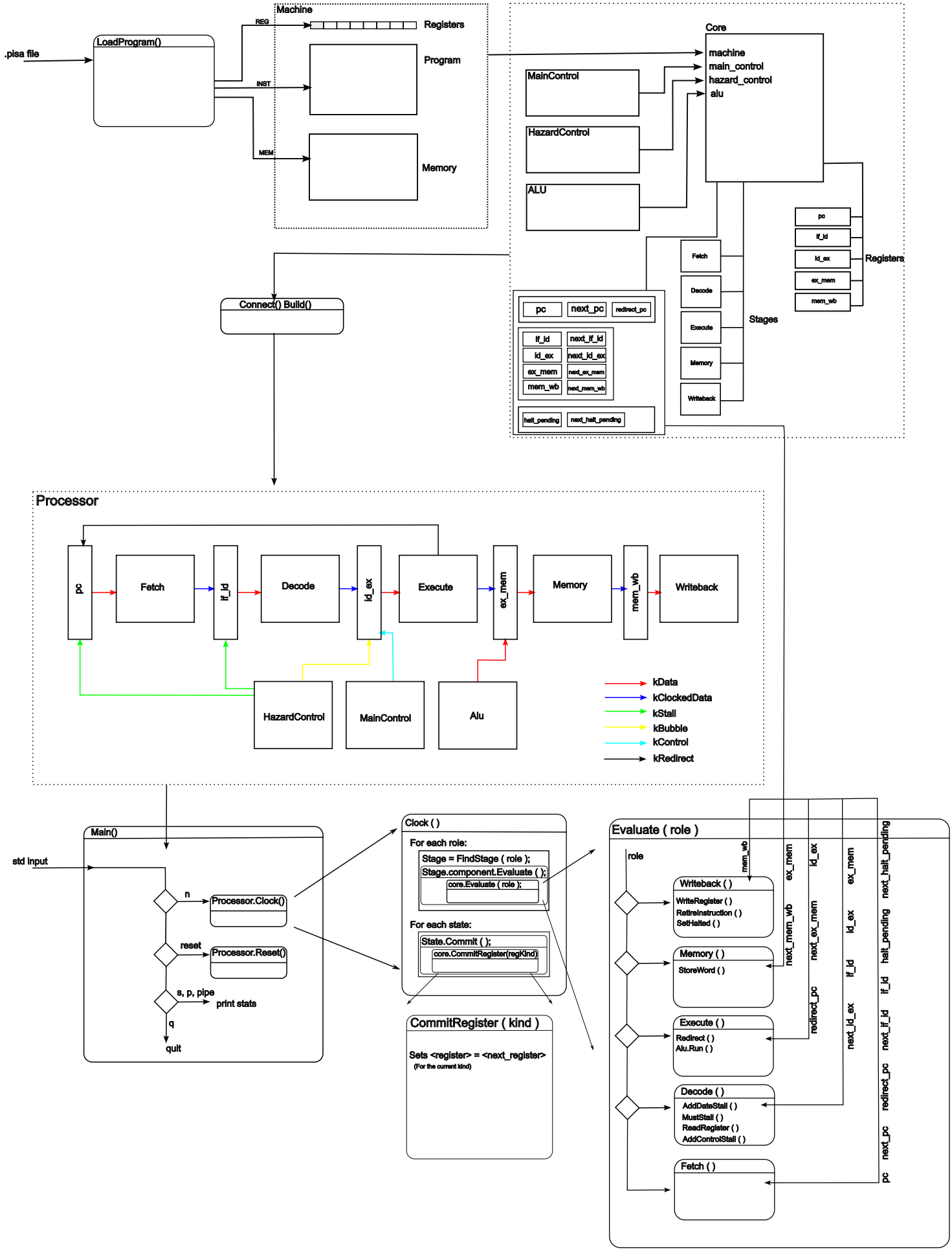
Once a control instruction enters the pipeline, fetch is held to prevent the processor from fetching or executing wrong-path instructions.

In EX, BEQ and BNE compare their two source operands, while J directly selects its target address. The branch target is taken from the instruction immediate, which represents an absolute byte address. After resolution, the processor redirects the PC to either the branch target or, for a not-taken conditional branch, to PC + 4.

The IF/ID latch is cleared during this redirect, ensuring that the next fetched instruction is from the correct path.

3 Diagram of Processor:

Below given is the complete flow of the program as well as the flow of data and control through processor. The diagram depicts what occurs when the program is run. Starts by creating the processor, connects all components, simulates the running of the input program and exits.


.pisa file
LoadProgram()
Machine
REG
INST
MEM
Registers
Program
Memory
Core
machine
main_control
hazard_control
alu
MainControl
HazardControl
ALU
pc
if_id
id_ex
ex_mem
mem_wb
Registers
Fetch
Decode
Execute
Memory
Writeback
Stages
pc
next_pc
redirect_pc
if_id
next_if_id
id_ex
next_id_ex
ex_mem
next_ex_mem
mem_wb
next_mem_wb
halt_pending
next_halt_pending
Connect() Build()
Processor
pc
Fetch
if_id
Decode
id_ex
Execute
ex_mem
Memory
mem_wb
Writeback
HazardControl
MainControl
Alu
kData
kClockedData
kStall
kBubble
kControl
kRedirect
Main()
std input
n
Processor.Clock()
reset
Processor.Reset()
s, p, pipe
print stats
q
quit
Clock ( )
For each role:
Stage = FindStage (role);
Stage.component.Evaluate ( );
core.Evaluate (role);
For each state:
State.Commit ( );
core.CommitRegister(regKind)
CommitRegister (kind)
Sets <register> = <next_register>
(For the current kind)
Evaluate (role)
role
mem_wb
ex_mem
id_ex
ex_mem
next_halt_pending
Writeback ( )
WriteRegister ( )
RetireInstruction ( )
SetHalted ( )
next_mem_wb
next_ex_mem
id_ex
halt_pending
Memory ( )
StoreWord ( )
redirect_pc
if_id
next_if_id
Execute ( )
Redirect ( )
Alu.Run ( )
next_id_ex
redirect_pc
Decode ( )
AddDateStall ( )
MustStall ( )
ReadRegister ( )
AddControlStall ( )
next_pc
pc
Fetch ( )

4 Cycle and Stall Counts Of Sample Test Programs:

Table 1 presents the measured cycle and stall counts for selected test programs. The processor uses a five-stage pipeline consisting of IF, ID, EX, MEM, and WB stages. Since forwarding is not implemented, instructions with read-after-write (RAW) dependencies must stall until the required value is written back to the register file.

Table 1: Cycle and stall counts for selected test programs

Test program	No. of Instr.	Cycles	Data stalls	Control stalls
`basic_alu.pisa`	8	14	2	0
`bne_taken.pisa`	6	13	2	1
`jump.pisa`	5	10	0	1
`raw_hazard.pisa`	4	12	4	0

basic_alu.pisa The program completed in 14 cycles with 2 data stalls. The `ADD` instruction uses registers `x1` and `x2`, whose values are produced by the preceding `ADDI` instructions. As there is no forwarding path, the pipeline stalls until these values are available in the register file during write-back.

bne_taken.pisa The program completed in 13 cycles with 2 data stalls and 1 control stall. The `BNE` instruction depends on the preceding `ADDI` results in `x1` and `x2`; this creates two RAW data-hazard stalls. Since `x1` and `x2` contain different values, the branch is taken. Fetch is held until the branch is resolved in the EX stage, producing one control stall. The instruction that would write `99` to `x3` is skipped.

jump.pisa The program completed in 10 cycles with no data stalls and 1 control stall. The unconditional `J` instruction has no register dependencies, so no data-hazard stalls occur. However, fetching is paused until the jump target is resolved, resulting in one control stall. The instruction at address `0x08` is skipped, and execution continues at address `0x0c`.

raw_hazard.pisa The program completed in 12 cycles with 4 data stalls. The first `ADD` depends on the value of `x1` produced by the preceding `ADDI`, causing two stall cycles. The following `ADD` depends on the value of `x2` produced by the first `ADD`, causing another two stall cycles. Thus, the two consecutive RAW dependencies produce a total of four data stalls. No branch or jump instruction is present, so there are no control stalls.

5 How to Run

Navigate to `assignment1` directory in `CS6600-18-PIPESIM`, and follow the commands to run it.

```
> make && ./pipesim example.pisa
```

These are the commands to use;

n advance one cycle.

n N advance up to N cycles, stopping early after `HALT` retires.

p : print registers, non-zero memory words, and halt status.

pipe : print the five stage snapshots after the last cycle.

graph : print the registered datapath/control graph and unit-use counts.

s : print cycle, retirement, and stall statistics.

reset : restore the initial program state.

q : exit.

To put a data value at some address before the program starts running, use a MEM record instead of INST

```
MEM 0x00000020 42
```

This just seeds that data memory before execution starts; it doesn't take up an instruction slot the way INST does.